Mini RAG Learning Lab

Building Retrieval-Augmented Generation from scratch —
PDF to vector search to grounded answer, with every number visible.

The complete pipeline, explained stage by stage, using the actual outputs from the executed notebook. Written for revision and interview preparation.

Ajmal Danish

github.com/AjmalDanish

1. What this project is

This is a single Google Colab notebook that implements a complete RAG (Retrieval-Augmented Generation) system for one small document — a company leave policy — without any high-level framework. The point was never to build a product; it was to understand, and be able to explain in an interview, exactly what happens between a PDF and an LLM answer.

RAG means three things: **R**etrieve relevant chunks of a document, **A**ugment the prompt with that retrieved context, and let the LLM **G**enerate an answer that is grounded in the context instead of its trained memory.

The notebook answers one question end-to-end, and every intermediate value is printed:

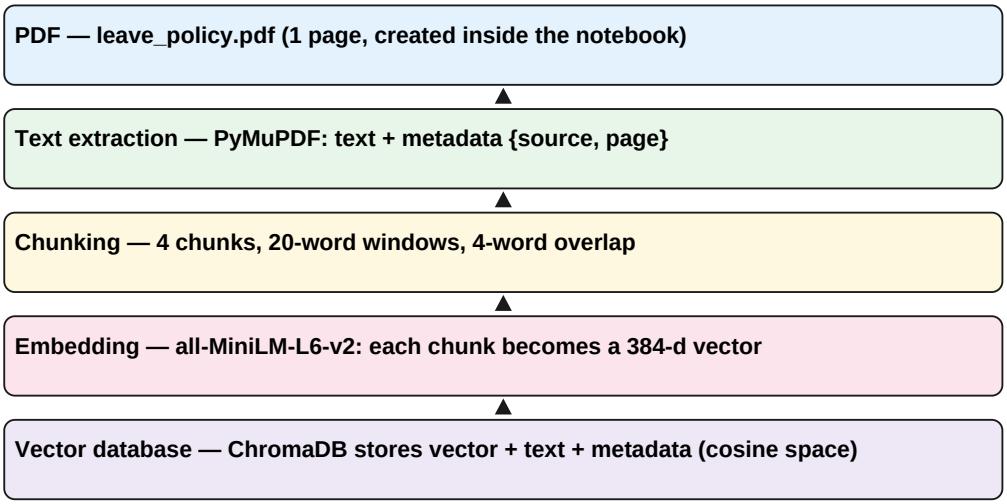
```
Question: How many weeks of maternity leave are available for a female worker? Answer:
26 weeks of paid maternity leave. Source: leave_policy.pdf — page 1
```

The answer above came from a real LLM (DeepSeek V4 Flash via OpenCode Go) that only saw the retrieved context — it did not read the PDF directly, and the question was answered with a citation back to the source document.

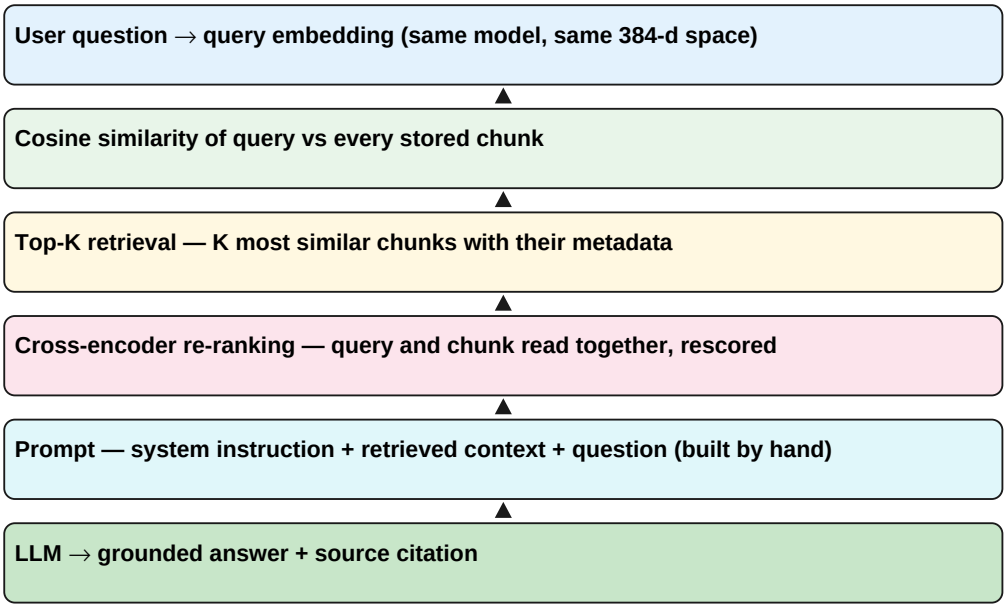
2. The pipeline at a glance

RAG has two separate phases. **Ingestion** runs once per document and prepares everything for search. **Query time** runs per user question.

Ingestion (offline)



Query time (online)



If the document changes, only ingestion needs to run again. If the question changes, only query time runs. Keeping these phases separate is one of the first things to say in an interview.

3. Stage by stage, with the real outputs

The rest of this document walks the pipeline in the same order as the notebook. All numbers shown here are the actual outputs of the executed notebook — nothing is invented. Toy examples used for teaching are labelled as such.

3.1 The document

The corpus is deliberately tiny so that every number is inspectable. The notebook creates the PDF programmatically:

```
COMPANY LEAVE POLICY Annual Leave: Employees receive 18 days of annual leave per year.
Sick Leave: Employees receive 10 days of paid sick leave per year. Maternity Leave:
Female employees receive 26 weeks of paid maternity leave. Paternity Leave: Employees
receive 7 days of paid paternity leave. Work From Home: Employees may work remotely up
to 2 days per week with manager approval.
```

3.2 Text extraction

A PDF is drawing instructions, not text. PyMuPDF reconstructs the characters and we attach metadata that will travel with the text through the whole pipeline — this is what makes final-answer citations possible:

```
{ "text": "COMPANY LEAVE POLICY\nAnnual Leave: ...", "metadata": {"source":
"leave_policy.pdf", "page": 1} }
```

3.3 Chunking

Embedding models have an input limit, and retrieval works better when each stored piece covers one idea. The notebook uses a plain-Python sliding window: 20 words per chunk, 4 words of overlap, so a phrase cut at a boundary still appears complete in at least one chunk. With this document that produces 4 chunks, e.g.:

```
Chunk: "Female employees receive 26 weeks of paid maternity leave." Overlap example: A B
C D E F G H / E F G H I J K L (E F G H repeated so no phrase is cut in half)
```

There is no universally optimal chunk size — too small loses context, too large dilutes similarity. The notebook tests both failure modes in Section 6 of this document.

3.4 Tokenization and token IDs

The tokenizer used is the real one that belongs to the embedding model (BERT-style WordPiece, vocabulary of 30,522 tokens). One important idea: a token ID is **not a meaning** — it is an integer index into the vocabulary, like a page number in a dictionary.

Token	Token ID	Note
female	2931	lowercased by the tokenizer
employees	5126	
receive	4374	
26	2656	numbers are tokens too
weeks	3134	
of	1997	
paid	3825	
maternity	23676	a single whole-word token
leave	2681	
'.'	1012	punctuation is a token

A word can split into several subword tokens — the real MiniLM tokenizer splits `unbelievably` into `['un','##bel','##ie','##va','##bly']`. The ## marks a continuation piece.

3.5 Token ID → embedding lookup

The embedding matrix E is a learned table: rows = vocabulary tokens, columns = embedding dimensions. Embedding a token is selecting one row. Mathematically this is a one-hot vector multiplied by E (a toy 5×3 example is used in the notebook, clearly labelled):

```
one_hot(4) = [0, 0, 0, 0, 1] one_hot(4) × E = 0·row0 + 0·row1 + 0·row2 + 0·row3 + 1·row4 = row4
```

Modern libraries do an efficient row lookup rather than building one-hot vectors — but the matrix-multiplication view is the correct mental model and makes the gradient math clean.

3.6 Where embedding values come from

Nobody ever decided that a dimension equals 0.17. The embedding matrix is a set of model parameters: initialized randomly, then updated by gradient descent during training so the numbers become useful for the task. The notebook runs this with real PyTorch autograd on a tiny 5×3 table:

```
E_new = E_old − learning_rate × dL/dE forward: ids [4, 1, 2] → mean-pooled prediction → MSE loss = 1.4338 backward: dL/dE computed by autograd update: E_after == E_before − 0.1 × grad (verified True)
```

Only rows that were used in the forward pass receive gradient — the printed gradient is zero for unused rows. This is the standard neural-network training rule, applied to the embedding table itself. No single dimension is labelled with a human concept; meaning is distributed across all dimensions, which is why similar words end up with similar vectors.

3.7 Inside the embedding model

The real model is **sentence-transformers/all-MiniLM-L6-v2**: 12 Transformer layers, 384-dimensional output. The chain inside it:

```
TEXT → token IDs → embedding lookup → token vectors (static, no context) → 12 Transformer layers of self-attention → contextual representations (each token influenced by all others) → mean pooling over tokens → L2 normalization → final 384-d vector
```

Self-attention is taught with a hand-built toy calculation in the notebook (3 tokens, 4→2 dims), printing Q, K, V, the scaled scores, the softmax weights (each row sums to 1), and the weighted sum:

```
Q = X·W_Q K = X·W_K V = X·W_V Attention(Q,K,V) = softmax( Q·K■ / √d_k ) · V
```

For the real model, the notebook opens the model's insides and reproduces mean pooling by hand from the last hidden state — the hand-made vector matches `model.encode()` to within `1.49e-08`, so the explanation is verified against the actual library output.

3.8 The real chunk embedding

```
input text : Female employees receive 26 weeks of paid maternity leave. embedding shape
: (384,) first 10 values : [0.0147, −0.0286, −0.0403, 0.0531, 0.0447, 0.0860, −0.0824,
−0.0629, −0.0483, 0.0374] L2 norm : 1.0 (normalized → only direction matters)
```

These 384 numbers are the sentence as far as the vector space is concerned. Dimension 7 does not 'mean' maternity — the meaning is distributed over all 384 dimensions.

3.9 Cosine similarity

Similarity is the cosine of the angle between two vectors. The notebook calculates it once by hand on a tiny 2D pair, then applies the identical formula to the real 384-d vectors:

```
cosine(q, c) = (q · c) / (■q■ · ■c■) hand example: q = [3, 4], c = [1, 2] dot = 11, ■q■
= 5, ■c■ = 2.236 → cosine = 0.9839
```

Real similarities of the five topical chunks against the maternity query:

Chunk	Cosine similarity
Maternity — “Female employees receive 26 weeks of paid maternity leave.”	0.8543
Paternity — “Employees receive 7 days of paid paternity leave.”	0.5477
Annual — “Employees receive 18 days of annual leave per year.”	0.4948
Sick — “Employees receive 10 days of paid sick leave per year.”	0.4223
WFH — “Employees may work remotely up to 2 days per week…”	0.3810

The maternity chunk wins by a clear margin, and semantically related chunks (Maternity ↔ Paternity, 0.611) are closer than unrelated ones (Maternity ↔ WFH, 0.414) — the model encodes meaning, not just word overlap.

3.10 Vector database

ChromaDB stores each chunk as (vector, text, metadata) and answers nearest-neighbour queries with approximate search. A key distinction for interviews: **the embedding model understands language; the vector database does not**. It is a fast warehouse of numbers. If the embedding model changes, everything must be re-embedded.

```
collection: leave_policy (4 vectors, cosine space) stored: chunk_00 [0.03, …] "COMPANY
LEAVE POLICY …" {source, page: 1} chunk_01 [-0.01, …] "Employees receive 10 days …"
{source, page: 1} …
```

3.11 Top-K retrieval

The query is embedded with the same model and compared against all chunks. Retrieval returns candidates — not the answer yet. With the full-document chunks (not the clean topical ones), the actual top-3 were:

Rank	Similarity	Chunk (truncated)
1	0.7293	Employees receive 10 days of paid sick leave per year. Maternity Leave: …

Rank	Similarity	Chunk (truncated)
2	0.7093	weeks of paid maternity leave. Paternity Leave: Employees receive 7 da…
3	0.4601	COMPANY LEAVE POLICY Annual Leave: Employees receive 18 days of annual…

3.12 Re-ranking with a cross-encoder

Bi-encoder retrieval compares two independently encoded vectors — fast, but coarse. A cross-encoder reads the query and chunk **together** through the Transformer, so it scores joint relevance much more precisely. It is too slow for the whole corpus, so it only re-scores the top candidates:

Cross-encoder scores (query, chunk)	Before (cosine)
7.2709 — sick + maternity chunk	0.7293 (rank 1)
5.0641 — maternity + paternity chunk	0.7093 (rank 2)
–2.5196 — annual-leave chunk	0.4601 (rank 3)
–3.0774 — work-from-home chunk	0.3633 (rank 4)

The re-ranker confirmed the retrieval order here, and in the failure experiments it visibly fixes cases where vector similarity alone ranks the wrong chunk first.

3.13 The prompt and the answer

The prompt is a plain string: system instruction + retrieved context + question. No framework builds it:

```
You are an HR assistant. Answer using ONLY the supplied context. If the answer is not present in the context, say you don't know. CONTEXT: [1] Employees receive 10 days of paid sick leave per year. Maternity Leave: Female employees receive 26 weeks of paid maternity … [2] weeks of paid maternity leave. Paternity Leave: … QUESTION: How many weeks of maternity leave are available for a female worker?
```

766 characters, ~173 tokens. Sent to DeepSeek V4 Flash through an OpenAI-compatible API. The final grounded answer, with its citation:

```
ANSWER: 26 weeks of paid maternity leave. Source: leave_policy.pdf — page 1
```

The system instruction plus the citation make this grounded generation — the standard answer to “how does RAG reduce hallucination?”.

4. Failure experiments

Six live experiments in the notebook show how RAG quality degrades, with the actual numbers from each run:

Experiment	What happened	Why it matters
Chunks of 3 words	best tiny chunk was a fragment like “26 weeks of” — sim 0.7221	chunks too small lose context
No overlap vs overlap at a phrase boundary	query “How many weeks of maternity leave?” scored 0.7221 without overlap, 0.8551 with	overlap keeps boundary phrases intact
Question the doc can't answer	all similarities ≈ 0.3 or below	low scores + “say you don't know” instruction stops hallucina
K = 1 for a two-fact question	only one fact retrievable	K is a recall/precision trade-off
K = 4 for a one-fact question	prompt grew 425 → 894 chars with irrelevant chunks	too much context adds tokens and noise
Re-ranking on “How much time off does a new parent get?”	cross-encoder reordered candidates vs pure cosine	joint query+chunk scoring catches paraphrase

5. RAG vs fine-tuning

	RAG	Fine-tuning
Where knowledge lives	External documents / vector DB	Inside model weights
Updating knowledge	Edit the document — instant	Re-train — hours/days
Best for	Factual, changing, verifiable knowledge	Style, tone, format, behaviour
Explainability	Citations possible	Weights are opaque
Failure mode	Retrieval misses or noise	Stale memorized facts

They complement each other: many production systems fine-tune for behaviour and use RAG for knowledge.

6. Interview Q&A;

Q: What is RAG, and why use it?

A: Retrieval-Augmented Generation: retrieve relevant document chunks, put them in the prompt, and let the LLM generate an answer grounded in that context. It gives up-to-date, domain-specific knowledge with citations, and reduces hallucination without expensive retraining.

Q: What is a token ID?

A: An integer index that identifies a token inside the tokenizer's fixed vocabulary (here 30,522 tokens). It is a lookup position, not a semantic value.

Q: Where do embedding values come from?

A: They are learned model parameters, optimized by gradient descent during training: E_new = E_old – lr·dL/dE. No human labels a dimension.

Q: What is an embedding?

A: A learned dense vector that represents text in a space where semantic similarity is measurable (e.g. via cosine). Here: 384-d, L2-normalized.

Q: What is cosine similarity?

A: (q·c)/(■q■·■c■) — the cosine of the angle between two vectors, in [–1, 1]. It ignores magnitude, which is why embeddings are normalized first.

Q: Why a vector database instead of NumPy?

A: Persistence, metadata filtering, and fast approximate nearest-neighbour search (ANN, e.g. HNSW) — brute-force comparison is O(N) per query.

Q: What does the vector DB understand about English?

A: Nothing. It stores and searches numbers. All meaning is created by the embedding model; change the model and you must re-embed everything.

Q: Why re-rank with a cross-encoder?

A: Bi-encoder retrieval is fast but coarse (two independent vectors compared). The cross-encoder reads query and chunk together and re-scores only the top candidates for precision.

Q: What is pooling, and which does MiniLM use?

A: Pooling turns a sequence of token vectors into one fixed vector. MiniLM uses mean pooling over real tokens plus L2 normalization — verified in the notebook against the library output.

Q: What is self-attention?

A: Each token gathers information from all tokens, weighted by compatibility: softmax(QK■/√d_k)·V. This is what makes representations contextual.

Q: What is chunk overlap for?

A: So phrases that straddle a chunk boundary appear complete in at least one chunk.

Q: How does RAG reduce hallucination?

A: Grounding: the model answers from supplied context under a “say you don't know” instruction, and the answer carries a source citation.

Q: RAG vs fine-tuning?

A: RAG injects editable, citable knowledge at query time; fine-tuning bakes knowledge into weights. RAG suits changing facts, fine-tuning suits behaviour.

Q: What are common RAG failure modes?

A: Bad chunk sizes, boundary splits, irrelevant retrieval, wrong K, and prompt noise — all demonstrated with real numbers in the notebook.

Q: Why embed the query with the same model as the documents?

A: Both must live in the same learned vector space, or similarity scores are meaningless.

7. One-page cheat sheet

```
INGESTION QUERY PDF question ↓ PyMuPDF extraction ↓ embed with the SAME model text +
metadata query vector ↓ chunking (20 words, 4 overlap) ↓ cosine similarity vs all chunks
chunks ↓ top-K retrieval ↓ tokenizer (vocab 30,522) ↓ cross-encoder re-ranking tokens →
token IDs ↓ best chunks ↓ embedding-matrix row lookup ↓ prompt (system + context +
question) token vectors ↓ LLM ↓ 12 Transformer layers ↓ answer + source citation
contextual token vectors ↓ mean pooling + normalize 384-d chunk embedding ↓ store in
ChromaDB
```

8. Glossary

Chunk	A short, self-contained text unit used for retrieval.
Token	The atomic text piece a model works on — a word or a subword.
Token ID	Integer index of a token in the vocabulary. A position, not a meaning.
Embedding matrix	Learned table (vocab_size × dim); each row is one token's vector.
Token embedding	The static dictionary vector of a token before seeing context.
Contextual representation	A token vector after attention has mixed in the other tokens.
Self-attention	Each token gathers information from all tokens, weighted by compatibility.
Q, K, V	Query / Key / Value projections in attention.
Pooling	Aggregating token vectors into one vector (MiniLM: mean pooling).
Chunk embedding	The fixed-size vector representing a whole chunk (384-d, normalized).
Cosine similarity	(q·c)/(■q■·■c■) — directional similarity in [–1, 1].
Vector database	Storage + fast nearest-neighbour search over vectors with payloads.
Top-K	The K highest-ranked candidates returned by retrieval.
Cross-encoder	Model that scores query+chunk jointly via full attention.
Re-ranker	Second-stage model that re-orders the candidate set precisely.
Grounding	Forcing the answer to be based on provided evidence, with citations.
Hallucination	The LLM producing plausible but unsupported text.

Source: the executed notebook Mini_RAG_Learning_Lab.ipynb — all numbers in this document are its actual outputs.